



THE CITY OF VAEL

FOUNDATION

02 – Data Model and Supabase Review

The blueprint is drawn. The building today still sits on compacted soil: each visitor's browser.

PREPARED FOR

Tony Grant, Owner
TSG Construction LLC

DOCUMENT TYPE

Milestone 1 deliverable
PRD acceptance pack

PREPARED BY

Appifest – Technical Audit
Continuing the existing prototype

DATE / METHOD

25 August 2026
Read-only inspection of the live tree

1. Two Foundations, one City

If you only remember one thing from this document: **the running City does not use the database drawings**. Those drawings are real, sequential, and carefully locked. They are not poured under the app the Owner can click today.

	LIVE PROTOTYPE (TODAY)	DRAWN FOUNDATION (SQL)
Where data lives	Browser <code>localStorage</code> via <code>mockX.ts</code>	Postgres tables in <code>supabase/migrations/0001 - 0020</code>
Shared between devices?	No	Would be, if applied and connected
Who can tamper	Anyone with the browser's storage	Row-level locks + privilege baseline (see 03)
Connected to UI?	Yes – this is the product	Providers exist; most screens do not call them

DOCS VS LEFTOVER LINK METADATA

`supabase/README.md` and the product README say: nothing applied, no project linked, runtime client is `null`. That last part is true in this tree – there is no `.env`, so `src/integrations/supabase/client.ts` exports `supabase = null`. Separate leftover CLI files under `supabase/.temp/` name a hosted development project `vael-media-tech-dev`. Migration comments 0017 and 0020 disagree with each other about how much of the schema was ever applied there (0017: only 0001/0002; 0020: 0001-0019). This audit did *not* connect to that project. Treating “runtime is mock” as fact, and “hosted leftover exists” as a Punch List item to reconcile with the Owner before any connection work.

2. Migration map (the drawings, in order)

#	PURPOSE
0001	Private <code>users</code> , <code>user_roles</code> , <code>auth</code> → <code>users</code> bootstrap trigger
0002	Public <code>profiles</code> ; authenticated-only <code>district_profiles</code>
0003	<code>visibility_plans</code> , <code>vaels</code> , <code>rael</code> lifecycle events
0004	Orphan <code>handshake_requests</code> ; real <code>connections</code> table
0005	Conversations and participants (no messages yet)
0006	Atomic Handshake/connection lifecycle functions
0007	Expiry processor for VAELs

#	PURPOSE
0008	First-admin bootstrap; lock sensitive columns
0009	Full role hierarchy + admin audit log
0010	Audit actor type (user / service / system)
0011	Notification events + unused deliveries table
0012	Notification preferences; SMS consent records
0013	Messages; attachment table reserved
0014	Media objects lifecycle
0015	Blocks, reports, moderation cases
0016	Profile documents + community posts/comments/reactions/saves
0017	Explicit table privileges so locks actually apply on both local CLI and hosted defaults
0018	Bugfix: ambiguous column in conversation ensure-function
0019	Owner can read admin notes on their own documents
0020	Strip service-role blanket table access; harden function execute

0001-0016 each still carry a header that says they are local design-only and not applied. 0017-0020 are written as corrections against a real development project. The drawings matured past the original “design only” stamp. The running app did not follow them.

3. Table inventory (29 tables)

All 29 tables are created in 0001-0016. Later files adjust privileges and functions; they do not add tables. RLS is enabled on every table. Most writes are not “update the row from the browser” – they go through named server functions.

TABLE	WHAT IT HOLDS	WHO MAY SEE IT (DESIGN)
users	Private account row (email, phone, status)	Self; Owner/Admin all. No client INSERT
user_roles	owner / admin / manager / developer / moderator / member	Self; Owner/Admin all. Writes via functions only
profiles	Username, display name	Public select
district_profiles	Per-district type + JSON payload	Signed-in only (Owner decision, ratified in 0019)

TABLE	WHAT IT HOLDS	WHO MAY SEE IT (DESIGN)
visibility_plans	Duration catalog; purchasable stays false	Public select; no client writes
vaels	Availability / need listings	Active rows public; owner sees all statuses
vael_lifecycle_events	Append-only history	Owner of that VAEL
handshake_requests	Early drawing, never used by product code	Requester or VAEL owner
connections	The real Handshake / match record (largest table)	The two parties only
connection_lifecycle_events	Append-only history	Parties
conversations / conversation_participants	Thread per connection	Participants
messages	Chat, including system notices	Participants; writes via <code>send_message()</code>
message_attachments	Reserved; client never reads them back	Participants
media_objects	File metadata; bytes not stored here	Owner or participant of linked message
notification_events	In-app notification records	Owner only
notification_deliveries	SMS/email/push log – no insert path exists	Owner via join
notification_preferences / sms_consent	Channel toggles; consent flags	Owner
blocks	Block list (soft-revoke in SQL)	Blocker only – blocked party sees nothing
reports / moderation_cases / moderation_actions	Trust & safety	Reporter or Owner/Admin/Moderator
admin_action_audit	Governance log	Owner/Admin; no TypeScript reader
profile_documents	License, insurance, capability statement, etc.	Owner, or public if flagged
community_posts / comments / reactions	Discussion	Public select
community_saves	Saved posts	Owner of the save only

4. How a person would get a Foundation row

Client INSERT into `public.users` is intentionally forbidden. Migration 0001 creates trigger `on_auth_user_created` on `auth.users` calling `handle_new_auth_user()`, which inserts the matching `public.users` row. A leftover commented duplicate of that trigger sits underneath – noise, not a second path.

A prior engineer note (`docs/audit/06-schema.md`) said this trigger was only a commented sketch. **That note is stale.** In the current 0001 file the trigger is active in the drawing. What remains true: the *running app* never creates `public.users` rows, because it never talks to this database. Nearly every other table foreign-keys to `users(id)`. Connecting screens to Supabase without this bootstrap working would fail the first real write.

5. Rooms drawn but not used

TABLE	WHY IT IS EMPTY IN PRACTICE
<code>handshake_requests</code>	0004 itself says foundation-only. Product folded handshake state into <code>connections</code> . Table was never dropped.
<code>notification_deliveries</code>	No function inserts a row. Waiting for a real SMS/email/push vendor (Change Order).
<code>user_roles</code> / <code>admin_action_audit</code>	No TypeScript governance module. Roles exist only in SQL.
<code>visibility_plans</code>	Catalog; client never selects it. Duration is also hardcoded in the app’s plan list.
<code>message_attachments</code>	SQL can attach; the messaging provider always returns <code>attachmentIds: []</code> .
<code>users</code>	No provider queries it. Identity goes through Auth + <code>profiles</code> .

6. Where the drawing is thinner than the prototype

If the City were cut over onto this Foundation as written, these prototype capabilities would silently shrink:

ROOM	PROTOTYPE KEEPS	FOUNDATION DRAWING DROPS OR CANNOT DO
Veil	Description, contact, requirements, budget, timeline	<code>vaeIs</code> has no columns for that rich content. Provider accepts it and never sends it.
Re-veil	Reconciles open connections after a new VAEL	Supabase <code>reconcileAfterRevail()</code> is a documented no-op (empty arrays).
Messages	Real attachment metadata	Always <code>[]</code> on the Supabase provider.

ROOM	PROTOTYPE KEEPS	FOUNDATION DRAWING DROPS OR CANNOT DO
Media	Instant local preview; status "stored"	Row stays <code>pending</code> forever. Nothing calls the service-role "stored" mark. No bucket.
Documents	Actual file bytes as a data URL	Metadata only. File bytes would vanish.
Auth admin	Local <code>isAdmin</code> flag	Hardcoded <code>false</code> in the Supabase auth provider.
Community	Seeded demo posts	No sample posts – by design.
Block history	Unblock deletes the row	SQL soft-revokes and keeps history (SQL is stricter here).

Generated TypeScript database types (`src/integrations/supabase/types.ts`) are still an empty stub. Providers cast past the type checker. A misspelled column would not fail Inspections; it would fail at runtime against a live project.

7. Live storage map (what actually holds water)

Twenty-four-plus keys, all device-local. See Document 01 for the ownership table. Nothing in `src/pages` should write these keys except through the owning module. Community, documents, and some profile writes now go through the service seam, which in local mode still ends in the same keys.

8. Cross-provider consistency

Re-checked against current providers: each table is owned by one domain's TypeScript provider (profile + profile-directory both read `profiles` / `district_profiles` with compatible partial shapes). No two providers fight over the same columns. Every `.from("...")` / `.rpc("...")` name in those providers resolves to a table or function the migrations create. `exchange_listings` in dead `src/data/listings.ts` is the exception – it is not in the Foundation at all.

9. Foundation verdict

BLUEPRINT QUALITY VS POUR STATUS

This is a serious Foundation drawing: 29 tables, function-mediated writes, privilege hardening through 0020, SQL tests sitting beside the migrations. It is not a live basement. Pouring it is a later, explicitly approved step – and it cannot be poured "as the product" until the field gaps in §6 are either added to the drawing or accepted as product reductions, and until screens are actually framed onto the seam (Document 01 §7).

THE CITY OF VAEL · Milestone 1 Technical Audit · 02 Foundation. This document does not authorize creating or linking a Supabase project.